

Proyecto de big data e inteligencia artificial

Fase 3

StockPulse

CURSO DE ESPECIALIZACIÓN EN

INTELIGENCIA ARTIFICIAL Y BIG DATA

MODALIDAD ONLINE

10/03/2026

Jimmy Chen

Joaquín Fernández

Sergio Romero

Deninson Tapia

Repositorio: https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5

1. Estado Final del Producto	4
1.1 Descripción general	4
1.2 Funcionalidades completadas	4
1.3 Mejoras respecto a Fase 2	4
1.4 Limitaciones conocidas	4
2. Mejoras Implementadas	4
2.1 Pipeline — Filtro de productos con pocas ventas (clean.py)	4
2.2 Pipeline — TRUNCATE en lugar de DROP TABLE (load_db.py)	5
2.3 Frontend — Refactorización y UX	5
2.4 Modelo ML — Mejora de precisión	5
3. Automatización del Sistema	6
Pasos ejecutados automáticamente	6
4. Despliegue y Justificación Técnica	7
4.1 Estado del despliegue	7
4.2 Arquitectura de despliegue actual	7
4.3 Decisión de no desplegar en producción	7
5. Arquitectura Final del Sistema	7
5.1 Diagrama de capas	8
5.2 Flujo de datos	8
6. Documentación Final	8
6.1 Guía de usuario	8
Iniciar el servidor	8
Iniciar sesión	8
Cargar el dataset	8
Consultar el dashboard	8
Interpretar los KPIs	8
Cerrar sesión	9
6.2 Guía técnica — Pipeline de datos	9
6.3 Instrucciones definitivas de ejecución	9
Paso 1 — Clonar el repositorio	9
Paso 2 — Crear entorno virtual	9
Paso 3 — Instalar dependencias	9
Paso 4 — Configurar credenciales	9
Paso 5 — Ejecutar el pipeline	9
Paso 6 — Arrancar el servidor	10
Paso 7 — Abrir el dashboard	10
7. Monitorización y MLOps	10
Implementado	10
Fuera del alcance	10
8. Repositorio Final	10
Estructura de ramas	10
9. Reflexión Final y Limitaciones	11
Logros destacados	11
Limitaciones asumidas	11

10. Presentación en Vídeo	11
Reparto de la presentación	11

1. Estado Final del Producto

1.1 Descripción general

StockPulse es un sistema de análisis y predicción de demanda en retail desarrollado sobre un stack Python full-stack. En su versión final permite cargar datasets de ventas, limpiarlos, almacenarlos en PostgreSQL (Supabase), entrenar un modelo Ridge y visualizar resultados en un dashboard web con autenticación.

1.2 Funcionalidades completadas

- **Pipeline de datos end-to-end:** ingesta CSV/XLSX, limpieza, normalización y carga en Supabase.
- **Modelo de predicción Ridge:** features temporales (lags, medias móviles, estacionalidad). Métricas: RMSE, MAE y R^2 .
- **API REST con FastAPI:** endpoints de autenticación, carga de dataset, KPIs y predicciones.
- **Dashboard web:** interfaz HTML/CSS/JS con inicio de sesión, ventas históricas y predicciones.
- **Base de datos Supabase:** accesible por todos los miembros del equipo de forma remota.
- **Autenticación:** contraseña hasheada con bcrypt y sesiones por token.

1.3 Mejoras respecto a Fase 2

Área	Fase 2	Fase 3
Pipeline	DROP TABLE + recrear en cada ejecución	TRUNCATE RESTART IDENTITY: esquema persistente
Calidad datos	Todos los productos incluidos	Filtro min_ventas=5: SKUs con <5 ventas excluidos
Frontend	Monolítico (un solo archivo JS)	Refactorizado en múltiples archivos
UX Dashboard	Cierre sesión en 2 clics	Cierre sesión en 1 clic
Modelo ML	Ridge básico	Ridge optimizado post-filtrado
Visualización	Gráfica sin indicador de precisión	Precisión del modelo en la gráfica

1.4 Limitaciones conocidas

- Dataset sin categorías: todos los artículos etiquetados como 'Sin categoría'.
- Modelo agrega predicción mensual; las predicciones por SKU se obtienen por cuota histórica.
- Sistema no desplegado en producción: ejecuta en local con Supabase como BD remota.
- Pipeline sin reentrenamiento automático periódico.

2. Mejoras Implementadas

2.1 Pipeline — Filtro de productos con pocas ventas (clean.py)

Se añadió un filtro en `split_productos_ventas()` que excluye productos con menos de 5 transacciones, reduciendo el ruido estadístico en las cuotas históricas.

```
conteo_ventas = df.groupby("id_producto").size().rename("num_ventas")
```

```
productos_validos = conteo_ventas[conteo_ventas >= min_ventas].index
df = df[df["id_producto"].isin(productos_validos)]
```

min_ventas=5 es ajustable sin modificar el orquestador. Elimina SKUs con historial insuficiente que distorsionan las predicciones.

2.2 Pipeline — TRUNCATE en lugar de DROP TABLE (load_db.py)

En Fase 2 create_tables() ejecutaba DROP TABLE CASCADE, destruyendo el esquema completo. Solución en Fase 3:

```
-- Paso 1: crear tablas solo si no existen
CREATE TABLE IF NOT EXISTS productos (...);
CREATE TABLE IF NOT EXISTS ventas (...);
CREATE TABLE IF NOT EXISTS predicciones (...);

-- Paso 2: vaciar datos preservando el esquema
TRUNCATE TABLE predicciones, ventas, productos RESTART IDENTITY CASCADE;
```

RESTART IDENTITY reinicia los contadores SERIAL. CASCADE vacía primero las tablas hijas respetando Foreign Keys. El esquema permanece intacto entre ejecuciones.

2.3 Frontend — Refactorización y UX

- División del frontend en múltiples archivos: app.js, session.js, CSS separado.
- Cierre de sesión corregido a un único clic.
- Modo oscuro implementado en el dashboard.
- Lista de productos muestra 'Lista vacía' cuando no hay datos.
- Gráficas visibles aunque estén vacías.
- Porcentaje de precisión del modelo visible dentro de la gráfica circular.

2.4 Modelo ML — Mejora de precisión

- Cambio de granularidad (diaria -> mensual): el requerimiento de negocio es predecir demanda mensual para reposición de inventario, no diaria. En load_and_aggregate() la serie se resampla a frecuencia MS (Month Start) sumando unidades vendidas y se descarta cualquier mes con cobertura < 25 días para no introducir un punto sesgado a la baja por meses incompletos. Así la métrica reportada (RMSE, MAE, R^2) coincide exactamente con el objetivo de negocio y el ruido diario (eventos puntuales, festivos, promociones) queda absorbido al sumar el mes.
- Corrección del data leakage: en una iteración intermedia se añadieron features como Δ_7 = ventas - lag_7 (incluía el target como entrada, el modelo podía recuperar ventas como $\Delta_7 + \text{lag}_7$) y media_7 con rolling sin shift (la ventana incluía el día actual). El R^2 sobre test subía prácticamente a 1.0, bandera roja en un modelo lineal sobre datos de retail. En Fase 3 se reescribe src/models/features.py para que toda feature derivada del target use shift(1) o lags >= 1: ninguna ventana incluye el mes que se intenta predecir.
- Set de features extendido (11 features, todas sin leakage): lags mensuales (lag_1, lag_2, lag_3), media móvil con shift (media_3 = ventas.shift(1).rolling(3).mean()), volatilidad reciente (std_3 con shift), momentum corto (diff_1 = lag_1 - lag_2), calendario (mes, trimestre, tendencia lineal) y codificación

ciclica del mes (mes_sin, mes_cos) para que un modelo lineal no interprete enero (1) y diciembre (12) como extremos lejanos. Se descartan delta_7, ratio_7_30 y los lags 28/60/90 de la iteracion previa: o introducian leakage o no son justificables con ~12 puntos mensuales (riesgo claro de overfitting).

- Tuneo del hiperparametro alpha: el valor fijo de Fase 2 se sustituye por GridSearchCV + TimeSeriesSplit sobre la rejilla [0.01, 0.1, 1, 3, 10, 30, 100, 300, 1000]. Con el leakage corregido, la validacion cruzada temporal devuelve un alpha legitimo adaptado al tamaño real del dataset en lugar de un valor arbitrario. Se conserva un fallback a alpha=10 cuando hay menos de 6 meses utiles, ya que en ese regimen la CV deja de ser fiable.
- Tratamiento de outliers (IQR): el multiplicador pasa de 1.5 a 3.0 (Tukey far-outliers). Con 12 puntos mensuales el limite estandar 1.5 recortaba picos legitimos de retail (noviembre y diciembre), perjudicando justo los meses de mayor interes para inventario. Se mantiene el cap (no eliminacion) de Fase 2 para no perder filas en una serie ya corta.
- Invariantes respecto a Fase 2: transformacion log1p del target para estabilizar la varianza, StandardScaler antes de Ridge (la regularizacion L2 es sensible a la escala) y mismo flujo load_and_aggregate -> prepare_data -> train_and_evaluate -> save_*. Las firmas publicas (build_features, tratar_outliers, aplicar_log, run_training) y los artefactos generados (ridge_model.pkl, scaler.pkl, predicciones_ridge.csv, predicciones_ridge.png) se conservan exactamente iguales, de modo que la integracion con run_pipeline.py permanece sin cambios.
- Limitaciones asumidas: 12 meses utiles dejan ~9 puntos tras aplicar lags y un split temporal 80/20 con 7 meses de train y 2 de test, por lo que las metricas tienen varianza alta y deben interpretarse como orientativas. La estacionalidad anual (lag_12) requeriria al menos 24 meses de historico, no disponibles en el dataset; se aproxima con las features mes y mes_sin/mes_cos.

3. Automatización del Sistema

El sistema cuenta con el orquestador run_pipeline.py que ejecuta el pipeline completo con un solo comando:

```
python src/pipeline/run_pipeline.py --input data/raw/online_retail.xlsx
```

Pasos ejecutados automáticamente

Paso	Script	Descripción
1	ingest.py	Carga el CSV/XLSX y valida las columnas requeridas
2	clean.py	Limpieza, normalización y filtro min_ventas=5
3	clean.py	Separación en tablas Productos y Ventas
4	run_pipeline.py	Guarda ventas_limpias.csv en data/processed/
5	load_db.py	Conecta a Supabase, TRUNCATE y carga productos+ventas
6	train_model.py	Entrena Ridge, guarda modelo (.pkl) y predicciones (.csv)
7	run_pipeline.py	Calcula cuotas históricas per-producto

Paso	Script	Descripción
8	load_db.py	Inserta predicciones per-producto en tabla predicciones

Arranque del servidor web:

```
# Desde src/backend/ con entorno virtual activado:
uvicorn main:app --reload --port 8000
# Dashboard: http://localhost:8000 | Login: admin / admin123
```

Justificación: el proyecto opera en entorno académico con Supabase como única dependencia externa. El arranque en dos comandos (pipeline + uvicorn) es suficiente y está completamente documentado.

4. Despliegue y Justificación Técnica

4.1 Estado del despliegue

El sistema no está desplegado en producción. Se ejecuta en local con la BD alojada en Supabase (PostgreSQL gestionado), lo que permite que cualquier miembro del equipo trabaje con los mismos datos sin servidor local.

4.2 Arquitectura de despliegue actual

Componente	Dónde se ejecuta	Tecnología
Pipeline de datos	Local (máquina del desarrollador)	Python 3.10 + Pandas
Base de datos	Nube — Supabase (PostgreSQL)	Supabase Free Tier
API REST	Local — puerto 8000	FastAPI + Uvicorn
Dashboard	Navegador local	HTML + CSS + JavaScript
Modelo ML	Local — src/models/saved/	Scikit-learn Ridge (.pkl)

4.3 Decisión de no desplegar en producción

- El dashboard llama a la API en localhost; moverla requeriría actualizar todas las URLs del frontend.
- Supabase ya actúa como capa de datos remota, cubriendo el requisito de accesibilidad compartida.
- El Free Tier de Render hiberna la instancia tras inactividad, añadiendo latencia inaceptable para una demo.
- El tiempo disponible se destina a mejoras funcionales con mayor impacto en la rúbrica.

Conclusión: la arquitectura actual (local + Supabase) cumple todos los requisitos funcionales. El despliegue completo es viable con Render o Railway y se plantea como mejora futura.

5. Arquitectura Final del Sistema

La arquitectura mantiene el diseño en capas de Fase 1 con las actualizaciones de Fase 2 (MySQL → Supabase) y las optimizaciones de Fase 3.

5.1 Diagrama de capas

CAPA	COMPONENTES	TECNOLOGÍA
Ingesta	ingest.py Validación de columnas	Python + Pandas + openpyxl
Procesamiento	clean.py Filtro min_ventas, split tablas	Python + Pandas
Almacenamiento	load_db.py — TRUNCATE + INSERT database.py	PostgreSQL (Supabase)
Orquestación	run_pipeline.py	Python argparse
Modelado ML	train_model.py — Ridge features.py	Scikit-learn + joblib
API REST	main.py — FastAPI routes (dashboard, auth)	FastAPI + Uvicorn
Visualización	index.html, app.js session.js, styles.css	HTML + CSS + JS

5.2 Flujo de datos

- 1 — Dataset raw: El Data Engineer carga el fichero de ventas de Kaggle
- 2 — ingest.py: Carga en DataFrame y valida columnas requeridas
- 3 — clean.py: Elimina nulos, duplicados, cancelaciones y filtra SKUs con <5 ventas
- 4 — load_db.py: TRUNCATE tablas → INSERT productos → INSERT ventas (lotes de 5.000)
- 5 — train_model.py: Agrega por día → log → Ridge → predicciones sobre 20% test
- 6 — run_pipeline.py: Distribuye predicciones por cuota histórica → INSERT predicciones
- 7 — FastAPI /api/dashboard/*: Sirve KPIs, ventas y predicciones al frontend
- 8 — Dashboard: El usuario visualiza ventas reales y predichas por producto

6. Documentación Final

6.1 Guía de usuario

Iniciar el servidor

Ejecutar uvicorn main:app --reload --port 8000 desde src/backend/. Abrir http://localhost:8000 en el navegador.

Iniciar sesión

Credenciales: usuario admin, contraseña admin123. El sistema genera un token de sesión.

Cargar el dataset

En la sección de carga, seleccionar online_retail.xlsx. El sistema ejecuta el pipeline completo automáticamente.

Consultar el dashboard

Seleccionar un producto del desplegable. Las gráficas muestran ventas históricas del último mes y predicciones.

Interpretar los KPIs

Las tarjetas superiores muestran ventas totales, unidades y número de productos. La gráfica circular muestra la precisión R².

Cerrar sesión

Clic en el botón Cerrar sesión (esperar a eliminación de datos).

6.2 Guía técnica — Pipeline de datos

El pipeline está dividido en cuatro módulos independientes:

Módulo	Archivo	Función principal	Entrada	Salida
Ingesta	ingest.py	load_dataset()	Ruta al archivo	DataFrame bruto
Limpieza	clean.py	clean_and_normalize()	DataFrame bruto	DataFrame limpio
Split	clean.py	split_productos_ventas()	DataFrame limpio	df_productos, df_ventas
Carga BD	load_db.py	create_tables()+insert()	DataFrames limpios	Supabase actualizado
Orquestador	run_pipeline.py	run_pipeline()	Ruta archivo	BD+modelo+predi cciones

Variables de entorno requeridas (environment/.env):

```
DB_HOST      = aws-0-eu-west-1.pooler.supabase.com
DB_PORT      = 6543
DB_NAME      = postgres
DB_USER      = postgres.nbamabcfxtcltbzxuncg
DB_PASSWORD  = 123Grupo5321
```

6.3 Instrucciones definitivas de ejecución

Paso 1 — Clonar el repositorio

```
git clone https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5.git
cd Proyecto_IA_BigData_Grupo5
```

Paso 2 — Crear entorno virtual

```
python -m venv venv
# Windows: venv\Scripts\activate | Linux/Mac: source venv/bin/activate
```

Paso 3 — Instalar dependencias

```
pip install -r environment/requirements.txt
```

Paso 4 — Configurar credenciales

```
cp environment/.env.example environment/.env
# Editar .env con las credenciales de Supabase
```

Paso 5 — Ejecutar el pipeline

```
python src/pipeline/run_pipeline.py --input data/raw/online_retail.xlsx
```

Paso 6 — Arrancar el servidor

```
cd src/backend
uvicorn main:app --reload --port 8000
```

Paso 7 — Abrir el dashboard

Navegar a <http://localhost:8000> | Login: admin / admin123

7. Monitorización y MLOps

Implementado

- **Logging por consola:** cada paso imprime logs con prefijo [INGESTA], [LIMPIEZA], [SPLIT], [DB], [PIPELINE] con contadores de filas y registros.
- **Métricas del modelo:** RMSE, MAE y R^2 calculados en cada ejecución. El R^2 se almacena como `confianza_modelo` en la tabla predicciones.
- **Persistencia del modelo:** serializado con joblib en `src/models/saved/ridge_model.pkl` junto con el `StandardScaler`.
- **CSV de predicciones:** exportado a `data/results/predicciones_ridge.csv` para auditoría.

Fuera del alcance

- Reentrenamiento automático periódico (requeriría cron o Airflow).
- Versionado del modelo con MLflow o DVC.
- Alertas automáticas por degradación del modelo.

8. Repositorio Final

Campo	Detalle
URL	https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5
Rama principal	main (código estable — entrega final)
Estructura	<code>src/pipeline/</code> · <code>src/backend/</code> · <code>src/frontend/</code> · <code>src/models/</code> · <code>environment/</code> · <code>docs/</code>
Tag final	v1.0.0
README	Instrucciones de instalación, configuración y ejecución actualizadas
.gitignore	Excluye: <code>.env</code> , <code>__pycache__</code> , <code>venv/</code> , <code>data/raw/</code> , <code>*.pkl</code>

Estructura de ramas

```
main      ← Versión estable (entrega final)
Backend   ← Pipeline de datos (Sergio Romero)
ML         ← Modelos y entrenamiento (Deninson Tapia)
Frontend   ← Dashboard e interfaz (Joaquín + Jimmy)
```

9. Reflexión Final y Limitaciones

StockPulse ha alcanzado todos los objetivos mínimos de Fase 0: pipeline funcional, modelo evaluado, base de datos estructurada y dashboard con autenticación. El proyecto demuestra la integración coherente de múltiples tecnologías del ecosistema Data/IA en un sistema end-to-end.

Logros destacados

- Pipeline modular y reutilizable con separación de responsabilidades clara.
- Migración documentada de MySQL/XAMPP a Supabase para accesibilidad remota del equipo.
- Modelo Ridge con transformación log y tratamiento de outliers, evaluado con métricas estándar.
- API REST con autenticación segura (bcrypt + tokens de sesión).
- Dashboard web funcional sin dependencia de librerías de terceros para la UI.

Limitaciones asumidas

- Dataset sin categorías de producto: todos etiquetados como 'Sin categoría'.
- Predicciones por cuota histórica: aproximación válida pero mejorable con modelos por SKU.
- Sin despliegue en producción: viable con Render/Railway como siguiente paso.
- Sin reentrenamiento automático del modelo.

10. Presentación en Vídeo

Reparto de la presentación

Miembro	Contenido que presenta
Joaquín Fernández	Presentación del proyecto, problema que resuelve, estructura del repositorio y guía de inicio.
Sergio Romero	Pipeline de datos, base de datos Supabase y demostración de ejecución
Deninson Tapia	Modelo de predicción Ridge, métricas y decisiones técnicas
Jimmy Chen	Dashboard: páginas, estadísticas, gráficas e inicio de sesión en vivo